Dialog

Vorlesung Künstliche Intelligenz

Ulrike Pado

Und über die Sprache hinaus?

- LLMs generieren immer Ausgabetext auf eine Eingabe und sind sich auch immer sicher, dass die Ausgabe korrekt ist
  (Konfidenzwerte der Generierung sind hoch…)
- Aber:
  - Informationen lagen ggf. im Training nicht vor -> Halluzinationen
  - Gewünschte Aktion ist nicht rein sprachlich (Timer setzen, Kalendereintrag machen, …)
  - Auf verschiedene Eingaben muss unterschiedlich reagiert werden

Informationen aktuell halten: RAG

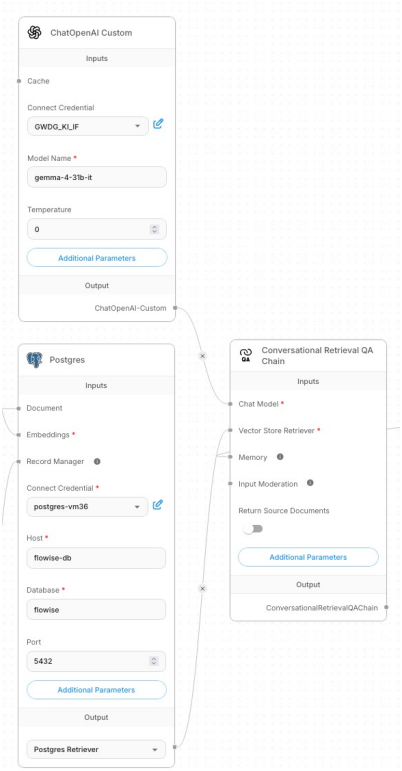
- Statt Nutzung impliziter Informationen, die beim Training gelernt wurden: Explizite Informationsquellen verwenden!
- Retrieval-augmented Generation: In Datenbank Informationen suchen, die zur Nutzeranfrage passen, diese als inhaltliche Grundlage für die Generierung von Output nutzen

  - Verringert Halluzinationen
  - Verringert die Chance, dass sich der Bot unerwünscht äußert
  - LLM formuliert nur noch gegebene Informationen um, nutzt nicht mehr primär sein inhärentes Wissens

RAG-Bot: SKIT, der KI-Tutorbot

LLM
(hier: Open Source-Modell Gemma)

DB-Komponente



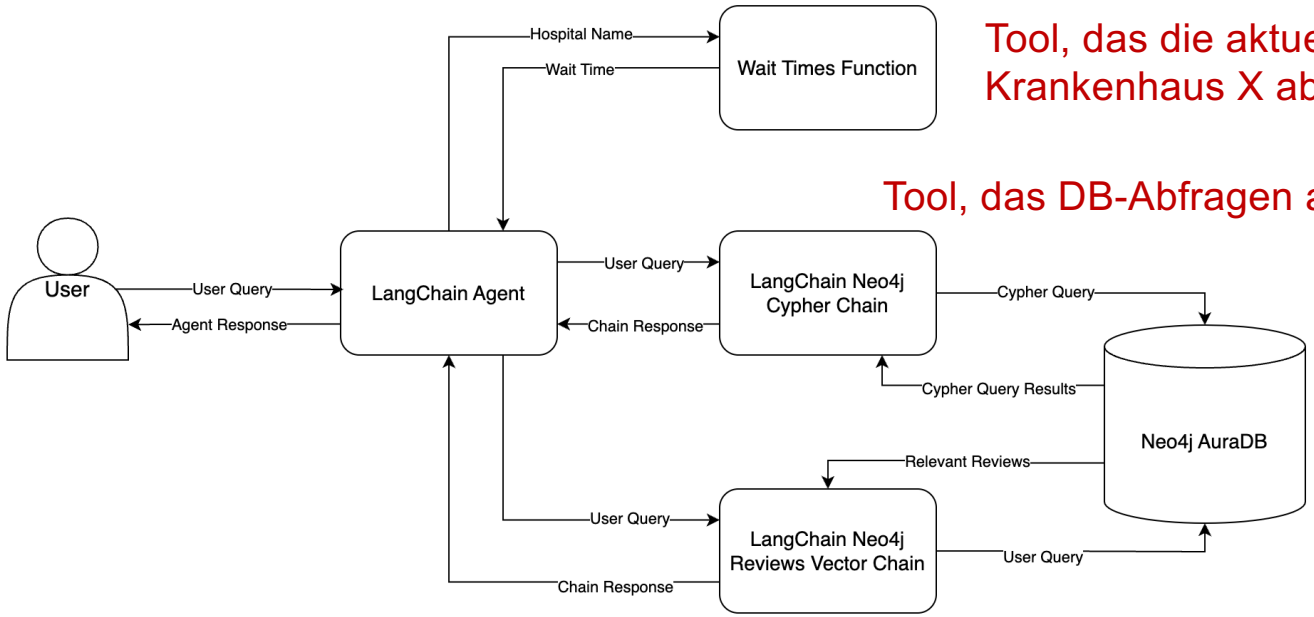
Agentenkomponente:

- Nutzeranfrage entgegen nehmen
- Maschinenlesbar codieren
- Relevante Informations-schnipsel aus der DB zur Antwortgenerierung an das LLM schicken

Agentische Architektur: Tools einbinden

- Zentrale Steuereinheit: Eine Agentenkomponente reagiert auf die Nutzereingabe
- Tools: von der Agentenkomponente auswählbare Tool-Implementationen (auf LLM-Basis oder direkt implementiert)
  - Internetsuche
  - DB-Abfrage
  - Kalendereintrag erstellen
  - …

Beispiel: Krankenhaus-LLM (Tutorial)



Tool, das die aktuelle Wartezeit in Krankenhaus X abfragt

Tool, das DB-Abfragen auf strukturierten Daten macht

Tool, das Sprachdaten aufgrund semantischer Ähnlichkeit zur Anfrage sucht und verarbeitet

Quelle: https://realpython.com/build-llm-rag-chatbot-with-langchain